NUTRITRACK

AI-Powered Personalized Nutrition and Wellness Tracking System

Project Documentation

Project Type: Full-Stack Web Application
Project Domain: Artificial Intelligence, Health and Wellness, Nutrition Tracking
Backend: Python Flask
Database: SQLite (Development)
ORM: Flask-SQLAlchemy
Authentication: Flask-Login
AI Integration: Ollama (Planned Integration)
Testing Framework: Pytest

---

1. INTRODUCTION

1.1 Project Overview

NutriTrack is a personalized nutrition and wellness tracking web application designed to help users monitor their daily food consumption, nutritional intake, hydration, and personal wellness goals.

The application provides a centralized platform where users can register, maintain their profiles, search for food items, record meals, calculate nutritional values, track water consumption, and monitor their daily progress.

The system uses a structured nutrition catalog to calculate calories and macronutrients based on the quantity of food consumed.

The proposed AI integration using Ollama will extend the application by introducing an intelligent nutrition assistant capable of interpreting user nutrition data, generating personalized food suggestions, explaining nutrition summaries, and answering general nutrition-related questions.

NutriTrack aims to make nutrition tracking accessible, convenient, and understandable through a combination of web technologies, relational database management, and artificial intelligence.

Disclaimer: NutriTrack is a general wellness and nutrition-tracking application. It is not intended to diagnose, prevent, or treat medical conditions or replace professional medical advice.

1.2 Problem Statement

Maintaining a balanced diet requires individuals to understand their food consumption, calorie intake, nutritional requirements, and hydration habits.

However, many individuals face difficulties because:

1. They do not maintain consistent records of their daily meals.
2. Calculating calories and nutritional values manually is time-consuming.
3. They may not understand the nutritional composition of different foods.
4. Tracking daily water consumption can be inconvenient.
5. They lack a centralized platform to monitor their nutrition history.
6. They may find it difficult to interpret their nutrition progress.
7. Existing manual tracking methods do not provide convenient personalized guidance.

Therefore, there is a need for a centralized digital platform that simplifies nutrition tracking and provides understandable insights into daily eating habits.

1.3 Proposed Solution

NutriTrack addresses these problems through a web-based nutrition management system.

The application provides:

- Secure user registration and authentication.
- Personalized user profiles.
- A searchable nutrition catalog.
- Meal and portion-based food logging.
- Automatic calorie and macronutrient calculations.
- Daily nutrition goal management.
- Water intake tracking.
- Daily nutrition dashboards.
- Rule-based wellness prompts.
- Configurable reminders.
- Restricted administrative functionality.

The planned Ollama integration will introduce AI-powered interaction and personalized nutrition assistance.

1.4 Project Objectives

The main objectives are:

1. Develop a user-friendly nutrition tracking application.
2. Automate calorie and macronutrient calculations.
3. Allow users to maintain daily meal records.
4. Provide personalized nutrition targets.
5. Monitor daily water consumption.
6. Present nutrition information through dashboards.
7. Maintain secure and user-specific records.
8. Implement modular software architecture.
9. Integrate local AI inference through Ollama.
10. Demonstrate practical applications of Python, Flask, SQL, database management, and artificial intelligence.

---

2. SCOPE OF THE PROJECT

2.1 Included Scope

The initial version of NutriTrack focuses on:

- User account management.
- Food catalog management.
- Meal tracking.
- Nutrition calculations.
- Water tracking.
- Nutrition goal management.
- Daily dashboard.
- Rule-based wellness prompts.
- Reminder management.
- Profile management.

- Administrator access control.

The proposed AI extension focuses on:

- AI nutrition assistance.
- Personalized food suggestions.
- Nutrition summary explanations.
- AI-assisted meal logging.
- General nutrition education.

2.2 Out of Scope

The following functionalities are not part of the initial MVP:

- Medical diagnosis.
- Disease detection.
- Medical treatment recommendations.
- Prescription diet generation.
- Clinical decision-making.
- Automatic medical record interpretation.
- Guaranteed calorie estimation from food photographs.

These features would require additional clinical validation, specialized datasets, and appropriate regulatory considerations.

---

3. TECHNOLOGY STACK

3.1 Backend Technologies

Technology	Version	Purpose
Python	3.x	Main programming language
Flask	3.1.2	Backend web framework
Flask-SQLAlchemy	3.1.1	Database integration and ORM
Flask-Login	0.6.3	User authentication and sessions
Pytest	8.4.2	Automated testing

3.2 Database

SQLite

SQLite is used as the development database.

It stores structured application information, including:

- User accounts.
- User profiles.
- Food catalog entries.
- Meal records.
- Meal items.
- Nutrition goals.
- Water logs.
- Reminder settings.

SQLAlchemy ORM provides an abstraction layer between the Flask application and the relational database.

3.3 Frontend Technologies

- HTML5: Application structure.
- CSS3: Styling and responsive layouts.
- JavaScript: Frontend interactions.
- Jinja2: Dynamic HTML rendering.

3.4 Artificial Intelligence

Ollama (Planned Integration)

Ollama will provide a local model inference environment.

It will allow the Flask application to communicate with locally running language models through HTTP APIs.

The proposed integration will support:

- Natural-language nutrition queries.
- Personalized food suggestions.
- Nutrition summary explanations.
- AI-assisted meal logging.

A suitable instruction-following model, such as Qwen3, can be evaluated according to available system resources.

3.5 Development Tools

- Visual Studio Code.
- Git and GitHub.
- PowerShell.
- Python Virtual Environment.
- Browser Developer Tools.

---

4. SYSTEM ARCHITECTURE

4.1 Architectural Approach

NutriTrack follows a **Modular Monolithic Architecture**.

The application is developed as a single Flask application, but its functionalities are separated into independent feature modules.

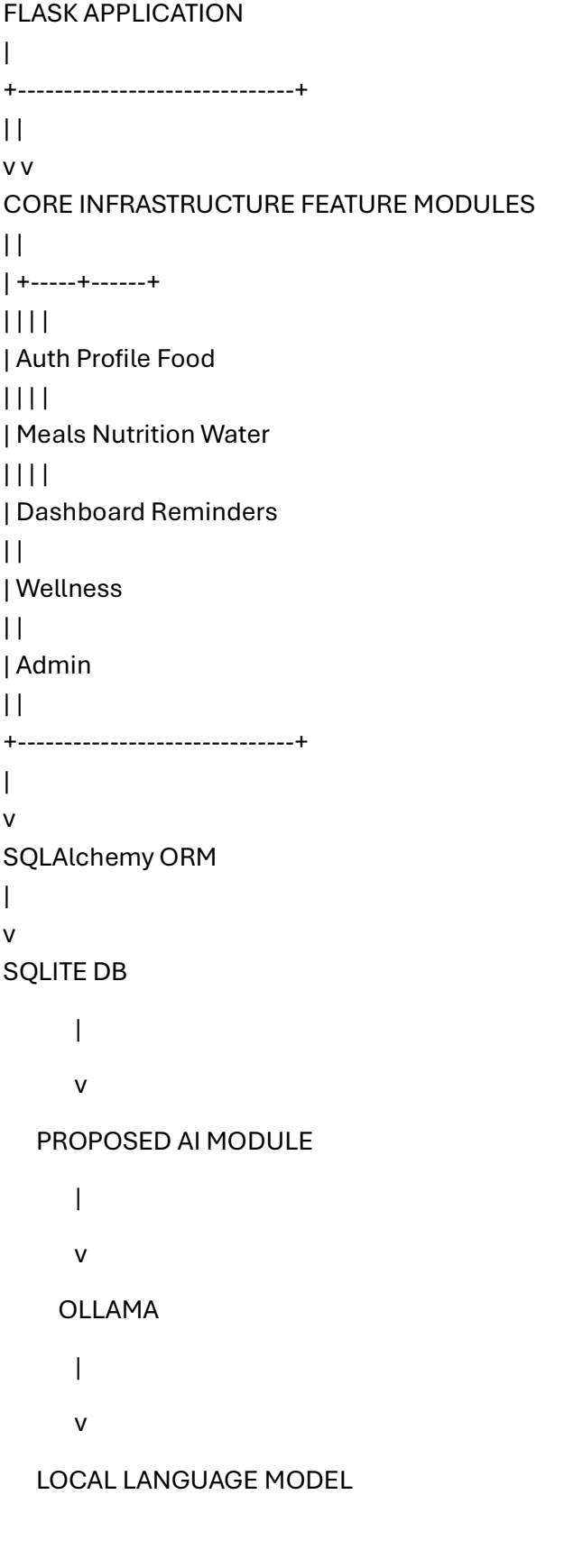
Each module manages its own routes and business logic while sharing common infrastructure.

This approach provides:

- Separation of responsibilities.
- Easier maintenance.
- Independent feature testing.
- Reduced code duplication.
- Controlled communication between modules.
- Simpler deployment compared with microservices.

4.2 High-Level Architecture

USER
|
v
FRONTEND INTERFACE
(HTML, CSS, JavaScript, Jinja2)
|
v


FLASK APPLICATION
CORE INFRASTRUCTURE FEATURE MODULES
Auth Profile Food
Meals Nutrition Water
Dashboard Reminders
Wellness
Admin
SQLAlchemy ORM
SQLITE DB
PROPOSED AI MODULE
OLLAMA
LOCAL LANGUAGE MODEL

The AI module will communicate with existing feature services to retrieve relevant user-specific nutrition information.

4.3 Architecture Components

Presentation Layer

Responsible for displaying application pages and receiving user input.

Application Layer

Contains Flask routes and coordinates communication between the frontend and business logic.

Business Logic Layer

Contains feature-specific services responsible for calculations, validation, and application operations.

Data Access Layer

Uses SQLAlchemy ORM to communicate with the database.

AI Integration Layer

Will manage prompts, context retrieval, model communication, and AI response handling.

---

5. FEATURE-WISE FUNCTIONAL REQUIREMENTS

5.1 Authentication Module

Description:

The authentication module manages user accounts and secure application access.

Functionalities:

- User registration.
- User login.
- Password hashing.
- Credential verification.
- Logout.
- Session management.
- Protected routes.

Workflow:

1. User opens the registration page.
2. User enters registration details.
3. The system validates the information.
4. Password is securely hashed.
5. User information is saved.
6. User logs in using valid credentials.
7. The system creates an authenticated session.

Expected Result: Only authenticated users can access protected application functionality.

5.2 Profile Management Module

Description:

Allows users to maintain their personal and dietary information.

Functionalities:

- View profile.
- Update profile.
- Store age, height, and weight.
- Manage dietary preferences.
- Manage nutrition-related preferences.

Workflow:

1. User opens the profile section.
2. User enters or updates personal information.
3. The system validates the information.
4. Updated details are stored in the database.

Expected Result: User-specific profile information is maintained.

5.3 Food Catalog Module

Description:

Maintains a structured collection of food items and their nutritional values.

Functionalities:

- Food search.
- Food retrieval.
- Food categorization.
- Nutrition information display.
- Administrator-controlled catalog management.

Workflow:

1. User opens food search.
2. User enters a food name.
3. The system searches the nutrition catalog.
4. Matching food items are displayed.
5. User selects the required food.

Expected Result: Users can retrieve nutritional information for available food items.

5.4 Meal Management Module

Description:

Allows users to record meals and specify food quantities in grams.

Functionalities:

- Meal creation.
- Food selection.
- Gram-based portion logging.
- Meal history.
- Meal editing.
- Meal deletion.
- Meal nutrient calculations.

Workflow:

1. User selects a meal category.
2. User searches for food.
3. User selects a food item.

4. User enters the quantity in grams.
5. The system calculates nutritional values.
6. The meal record is saved.
7. Updated totals are displayed.

Expected Result: Users can maintain accurate records based on the nutrition data available in the catalog.

5.5 Nutrition Calculation Module

Description:

Calculates the nutritional values of recorded food portions.

The current MVP implements a 100-gram calculation basis.

Formula:

Nutrient Value =
Food Nutrient Value × Quantity in Grams / 100

Example:

If 100 grams of a food contains 200 kcal and the user consumes 150 grams:

Calories = 200 × 150 / 100

Calories = 300 kcal.

The same calculation method is applied to protein, carbohydrates, and fat.

Values are rounded to two decimal places using half-up rounding.

Daily meal totals are calculated from persisted meal items.

Current Limitation:

Serving-unit conversion is not yet implemented. The initial implementation supports quantities in grams.

5.6 Nutrition Goals Module

Description:

Allows users to establish and monitor daily nutrition targets.

Functionalities:

- Daily calorie goals.
- Protein targets.

- Carbohydrate targets.
- Fat targets.
- Goal progress calculation.
- Remaining nutrition calculation.

Workflow:

1. User opens nutrition goals.
2. User selects or updates targets.
3. The system stores the targets.
4. Meal records are compared with the selected goals.
5. Progress is displayed.

5.7 Water Tracking Module

Description:

Allows users to record their daily water consumption.

Functionalities:

- Water logging.
- Daily water totals.
- Water goal management.
- Remaining intake calculation.
- Water history.

Workflow:

1. User opens the water tracker.
2. User selects the consumed quantity.
3. The system saves the water record.
4. Daily water totals are calculated.
5. Progress is displayed.

5.8 Dashboard Module

Description:

Provides a centralized summary of the user's nutrition activities.

Functionalities:

- Daily calorie summary.
- Macronutrient summary.
- Meal history.
- Water intake summary.
- Nutrition goal progress.
- Wellness prompts.

The dashboard retrieves relevant information from other feature modules.

It does not independently own meal or water records.

5.9 Reminder Module

Description:

Allows users to configure reminders related to meals and water consumption.

Functionalities:

- Reminder creation.
- Reminder editing.
- Reminder activation.
- Reminder deactivation.
- Reminder schedule management.

Actual scheduled notification delivery requires a scheduler or notification integration.

5.10 Wellness Module

Description:

Provides rule-based prompts based on recorded user activity.

Functionalities:

- Tracking-related prompts.
- General wellness messages.
- Nutrition goal progress notifications.
- Food-related educational information.

These prompts are intended for general wellness support, not medical diagnosis.

5.11 Administrator Module

Description:

Provides restricted administrative functionality.

Functionalities:

- Restricted administrator dashboard.
- Food catalog management.
- Catalog validation.
- Authorized administrative operations.

Access is restricted to users assigned the admin role.

---

6. PROPOSED AI INTEGRATION USING OLLAMA

6.1 Introduction

The proposed AI extension will integrate Ollama with the Flask backend to introduce intelligent nutrition assistance.

Ollama will run a language model locally and expose an HTTP API through which Flask can submit prompts and receive generated responses.

The AI module will remain independent of the existing nutrition modules.

6.2 Why Ollama?

Ollama is selected because:

1. It supports running language models locally.
2. It provides an HTTP API for application integration.
3. It allows evaluation of different open-weight models.
4. It can operate without sending every prompt to a hosted AI API.
5. It integrates with Python applications